

Controlling Processor C-State Usage in Linux

& Disabling C-States for Low Latency (limit c-state to c1 only!)

Introduction:

What are C-states, cstates, or C-modes?

There are various power modes of the CPU which are determined on the basis of their current usage and are collectively called “C-states” or “C-modes.”

C-states are idle power saving states, during a C-state (other than C0, which is the normal CPU operating mode, i.e., the CPU is 100% turned on), the processor is idle, meaning that nothing is executing

C-States range from C0 to Cn. C0 indicates an active state. All other C-states (C1-Cn) represent idle sleep states where the processor clock is inactive (cannot execute instructions) and different parts of the processor are powered down. As the C-States get deeper, the exit latency duration becomes longer (the time to transition to C0) and the power savings becomes greater.

- Each state of the CPU utilises different amount of power and impacts the application performance differently.
- Whenever a CPU core is idle, the builtin power-saving logic kicks in and tries to transition the core from the current C-state to a higher C-state, turning off various processor components to save power

But you also need to understand that every time an application tries to bind itself to a CPU to do some task, the respective CPU has to come back from its "deeper sleep state" to "running state" that needs more time to wake up the CPU and be again 100% up and running.

(This increases latency to all applications! and may generate instability in some use cases)

- The higher the C number, the CPU sleep mode is deeper, i.e., more circuits and signals are turned off and more time the CPU will require to return to C0 mode, i.e., to wake-up.

(The higher the C number / Deeper idle state, The greater the latency to return to C0, 100% operational cpu state)

Each mode is also known by a name and several of them have sub-modes with different power saving – and thus wake-up time – levels.

Below is a table explaining all the CPU C-states and their names:

mode	Name	What it does
C0	Operating State	CPU fully turned on
C1	Halt	Stops CPU main internal clocks via software; bus interface unit and APIC are kept running at full speed
C1E	Enhanced Halt	Stops CPU main internal clocks via software and reduces CPU voltage; bus interface unit and APIC are kept running at full speed
C1E	—	Stops all CPU internal clocks
C2	Stop Grant	Stops CPU main internal clocks via hardware; bus interface unit and APIC are kept running at full speed
C2	Stop Clock	Stops CPU internal and external clocks via hardware
C2E	Extended Stop Grant	Stops CPU main internal clocks via hardware and reduces CPU voltage; bus interface unit and APIC are kept running at full speed
C3	Sleep	Stops all CPU internal clocks
C3	Deep Sleep	Stops all CPU internal and external clocks
C3	AltVID	Stops all CPU internal clocks and reduces CPU voltage
C4	Deeper Sleep	Reduces CPU voltage
C4E/C5	Enhanced Deeper Sleep	Reduces CPU voltage even more and turns off the memory cache
C6	Deep Power Down	Reduces the CPU internal voltage to any value, including 0 V
C7	Deep Energy Saving	The CPU tries to flush its L3 cache. If the L3 cache <u>is able to</u> be entirely cleared, the CPU cuts its power to save energy. The power from the system agent is removed too.
C7s	—	When an MWAIT(C7) command is issued with a C7s sub-state hint, the entire L3 cache is flushed in one step as opposed to flushing the L3 cache in multiple steps. This also allows the system to send I/O devices to low power mode to reduce unnecessary power consumption when the system idles down.
C8	—	The L3 cache is flushed in a single step. The power to the PLL is cut.
C9	—	The VCCIN (VCC Input Voltage) gets lowered to a minimum.
C10	—	The single-phase core management system, VR12.6, goes into a low-power state. The CPU is almost shut down.

Latency sensitive applications do not want the processor to transition into deeper C-states, due to the delays induced by coming out of the C-states back to C0.

These delays can range from hundreds of microseconds to milliseconds. (& may generate instability)

If maximum overall performance is desired, all power-saving states can be disabled. Although this configuration minimizes the latencies to transition into and out of the various power states, energy use is increased. At the other extreme, if power settings are optimized for maximum energy savings, performance can suffer because of long latencies.

For most applications, the default system settings offer a good balance between performance and efficiency. On most Systems, the defaults can be changed if either increased performance or power savings is preferred.

Modern operating systems now include the ability to monitor, manage and report system behavior. Depending on OS kernel boot parameters, some OSes can ignore set BIOS/UEFI values (including C-states) and assign values according to installed application performance guidelines. They can also ensure adherence to latency limits and flag system issues when a system is unable to operate within those limits.

Important points:

- ▶ New hardware and software technology releases can add or change C-states selections
- ▶ Failure to understand the effects of the new values can result in unexpected system behavior, such as reboots, PSOD, BSOD, and hangs
- ▶ Newer generations of Hardware/Software may behave differently than previous generations with respect to C-states.
- ▶ Different generations of Hardware/Software may have different default settings.

Check which CPU idle driver is being used

Modern Linux kernels have Two built-in CPU idle drivers in the x86_64 architecture:

1: `intel_idle` : which is the one used by default and which will ignore any C-State limits imposed by Basic Input/Output System (BIOS)/Unified Extensible Firmware Interface (UEFI).

This driver may cause issues by enabling C-States even though they are disabled in the BIOS or UEFI. This can cause minor latency as the CPUs transition out of a C-State and into a running state.

2: `acpi_idle` : The `acpi_idle` cpuidle driver retrieves available sleep states (C - states) from the ACPI BIOS tables. If `intel_idle` is disabled, this causes `acpi_idle` cpuidle driver to be used.

****(it is heavily advised to not use `acpi_idle` driver and not to completely disable `intel_idle` driver c states, just limit `intel_idle` to max c state of 1 !!)**

Why the OS might ignore BIOS settings?

- The OS might ignore BIOS settings based on the idle driver which is in use.
- If one uses `intel_idle` (the default on intel machines) the OS can ignore ACPI and BIOS settings, i.e. the driver can re-enable the C-states.
- In case one disables `intel_idle` and uses the older `acpi_idle` driver the OS should follow the BIOS settings.

Determine what idle driver Linux is using:

```
$ cat /sys/devices/system/cpu/cpuidle/current_driver
```

Check Maximum C-State value Allowed for my CPU

We have a few methods of checking the allowed C-States for our CPU, Following will be listed 2 methods;

Check Maximum C-State value Allowed for my CPU: Method 1

```
$ cat /sys/module/intel_idle/parameters/max_cstate
```

Check Maximum C-State value Allowed for my CPU Using cpupower (linux-cpupower package): Method 2

```
$ cpupower idle-info
```

Disable C-States / Edit Maximum Allowed C-State Value

****Note: You Should Not actually disable c-states, but limit the max allowed c-state to 1 !**

c-state drivers/settings are not modular (compiled into your kernel and not a separate module), which means that the only way to pass configuration parameters to it is via the kernel command line parameters. *Although Dynamic control of c-states is also possible

****** By Default intel_idle is used: This driver uses knowledge of the various CPUs to control C-states without input from system firmware (BIOS). This driver will mostly ignore any other BIOS setting and kernel parameters, so if you want Absolute control over C-states, you should use kernel parameter "intel_idle.max_cstate=0"

(NOTE Just USE intel_idle.max_cstate=1)** to disable this driver. Note that the intel_idle driver will only take control for CPUs it specifically knows about, so if you have, for example, a very new system with an older linux Kernel, this may not be relevant.

You might also need to disable all c-states in BIOS setup, if you are looking for low latency.

When the intel_idle driver is disabled, the Linux kernel will use the acpi_idle driver to use C-states. System firmware (BIOS) provides a list of available C-states to the operating system using an ACPI table. So, once intel_idle is disabled, users who want low latency can disable C-states by going into system (BIOS) setup during boot and telling BIOS to disable C-states (which takes them out of the ACPI tables). Disabling C-states in this way will typically result in Linux using the C1 state for idle processors, which is fairly fast.

There are various methods to achieve this.

Method 1 (Preferred,Best way)

- ▶ Edit /etc/default/grub and add `intel_idle.max_cstate=1` parameter to the kernel line.
- ▶ update-grub

```
$ pluma /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="intel_idle.max_cstate=1"

$ update-grub
```

This disables C-states on kernel level.

****intel_idle.max_cstate=X** corresponds to intel_idle driver
processor.max_cstate=X corresponds to acpi_idle driver (BIOS), used if **intel_idle.max_cstate=0**

Method 2 This disables C-states on kernel level.

```
intel_idle.max_cstate=0
```

This might improve performance only when C-states for CPUs are disabled in the BIOS. If C states are enabled in the BIOS, you might still see performance issues. If that is the case, either disable them in the BIOS or **additionally** use the following kernel parameter:

```
processor.max_cstate=0
```

Method 3 Using cpupower linux-cpupower package

This method Gives us one interesting feature: it does not rely on GRUB, The cpupower command allows inspection and control of cpufreq and cpuidle tunables for hardware that support these features.. linux-cpupower package

cpupower

Available and used C-states can be shown with:

```
$ cpupower idle-info
```

To Disable a specific C-State number:

```
$ cpupower idle-set -d X <C-state no. X gets disabled (not used by the kernel anymore) at runtime>
```

To Enable a specific C-state number:

```
$ cpupower idle-set -e X <C-state no. X gets enabled (used by the kernel) at runtime>
```

Watch usage and available C-states in real time:

```
$ watch cpupower idle-info
```